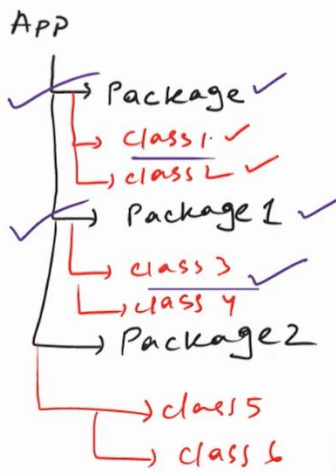




*** Access Modifier

	with in class	within Package	outside Package by Subclass	outside Package
Private	Yes	No	No	No
Default	Yes	Yes	No	No
Protected	Yes	Yes	Yes	No
Public	Yes	Yes	Yes	Yes ✓

class 1
 {
 m1
 m2
 }

Class

↳ block of code

↳ Property
 ↳ method.

↳ Passive Communicative system.

```
#include <stdio.h>
int main()
{
    int a, b, c;
    a = 5;
    b = 10;
    c = a + b;
    printf("%d", c);
    return 0;
}
```

class-1

```
class Calculator
{
    public static void main
    (String[] args)
    {
        A
    }
}
```

class-2

```
class Addition
{
    int a;
    int b;
}
public int sum()
{
    int c;
    c = a + b;
    return c;
}
```

```
#include <stdio.h>
int main()
{
    int a, b, c;
    a = 5;
    b = 10;
    c = a + b;
    printf("%d", c);
    return 0;
}
```

class-1

```
class Calculator
{
    public static void main
    (String[] args)
    {
        int nes;
        Addition objAdd =
        new Addition();
        objAdd.a = 5;
        objAdd.b = 10;
        nes = objAdd.sum();
        System.out.println(nes);
    }
}
```

class-2

```
class Addition
{
    int a;
    int b;
}
public int sum()
{
    int c;
    c = a + b;
    return c;
}
```

If you not initialize the objAdd.a then it automatically assign it objAdd.a=0;

In c it will garbage value. In java it is 0;

Java initialize 0 or Null.

Handwritten code for a Java class named `Example` illustrating a default constructor.

```
Class Example
{
    public static void main (String [] args)
    {
        Example obj = new Example();
    }
}
```

Annotation: Default constructor

```
Class Example
{
    public static void main
        (String [] args)
    {
        Example obj = new
            Example();
    }
}

Example()
{
    System.out.println("xyz");
}
```

Annotation: xyz

Default Constructor:

Handwritten code for a Java class named `Addition` illustrating a parameterized constructor.

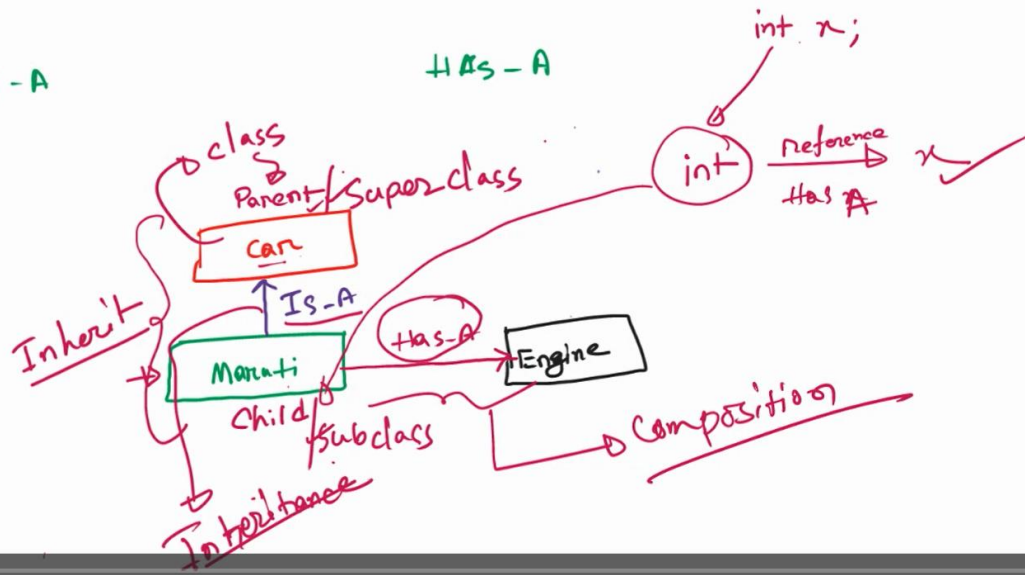
```
Parameterized constructor

Class Addition
{
    public static void main (String [] args)
    {
        Addition Add = new Addition( )
    }

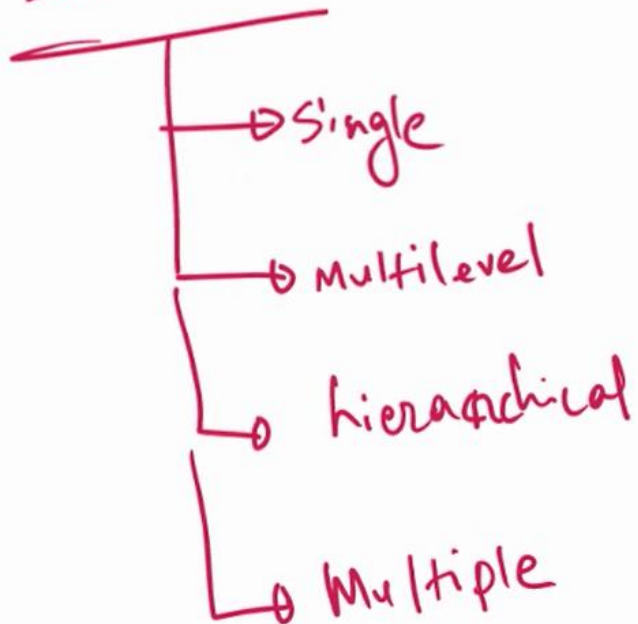
    Addition (int x, int y)
    {
        int z = x + y;
        System.out.println(z);
    }
}
```

IS-A

Has-A

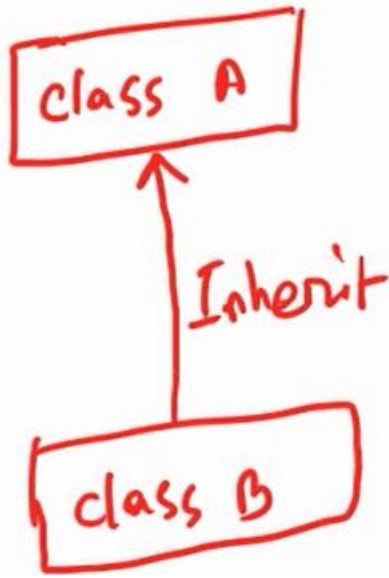


Inheritance



Single Inheritance :

Single Inheritance:



Class A (Parent class).

Class B(child class).

Dog is a animal.

Is a relationship.

Animal is a parent class.

Dog is a child child.

Why?

Dog is type of animal. IT depends on which is a type here.

So animal is parent. More example: Rasel is a Police. Similarly Police is a parent class. Rasel is a child class.

Sure. Let's make single-level inheritance as simple as possible.

✦ Upgrade

What is Single-Level Inheritance?

When one child class inherits from one parent class, it is called single-level inheritance.

Parent Class
↓
Child Class



For example:

Animal
↓
Dog



Dog inherits properties and methods from Animal.

1. Parent class — Animal.java

```
</> Java
package stcode;

public class Animal {

    String name;

    void eat() {
        System.out.println(name + " is eating.");
    }
}
```



2. Child class — Dog.java

✦ Upgrade  Share

```
Java   
  
package stcode;  
  
public class Dog extends Animal {  
  
    void bark() {  
        System.out.println(name + " is barking.");  
    }  
}
```

Notice:

```
Java   
  
extends Animal
```

This means:

`Dog` inherits from `Animal`.

Therefore, `Dog` can use `name` and `eat()` even though they are defined in `Animal`.



 Java



```
extends Animal
```

This means:

`Dog` inherits from `Animal`.

Therefore, `Dog` can use `name` and `eat()` even though they are defined in `Animal`.

3. Main class — `Main.java`

 Java



```
package stcode;

public class Main {

    public static void main(String[] args) {

        Dog d1 = new Dog();

        d1.name = "Tommy";

        d1.eat();
        d1.bark();
    }
}
```



Output

```
Tommy is eating.  
Tommy is barking.
```



How it works

When you write:

```
</> Java  
Dog d1 = new Dog();
```



you create an object of the **child class**.

Because:

```
</> Java  
public class Dog extends Animal
```



`Dog` gets the members of `Animal`.

So you can do:



```
</> Java  
d1.name = "Tommy";
```



and:

```
</> Java  
d1.eat();
```



even though `name` and `eat()` are written in `Animal`.

And:

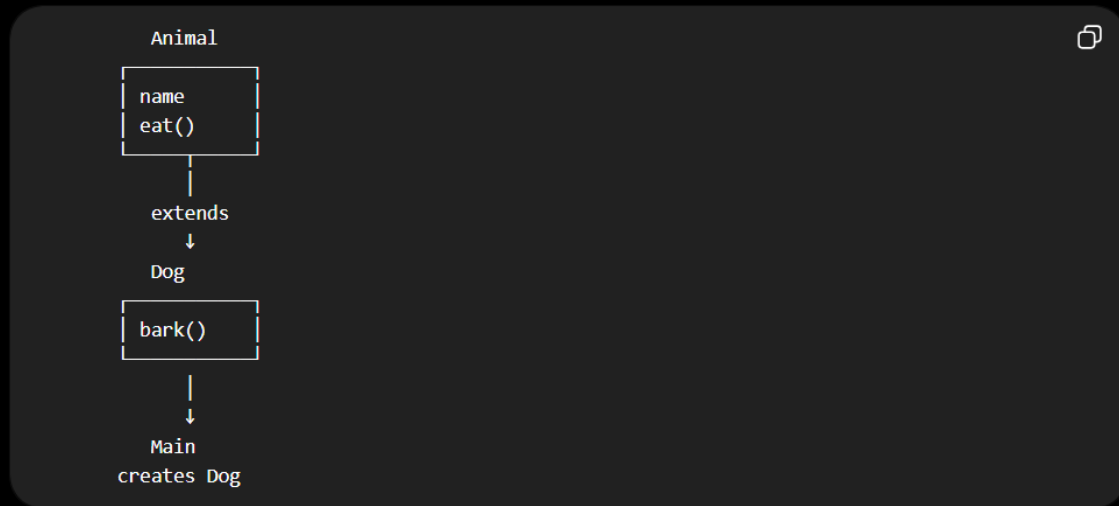
```
</> Java  
d1.bark();
```



works because `bark()` belongs to `Dog`.

works because `bark()` belongs to `Dog`.

In one picture



This is the basic form of single-level inheritance.

Multilevel Inheritance: Dada er sompotii silo 2 bigha, baba kichu loss kore nai.

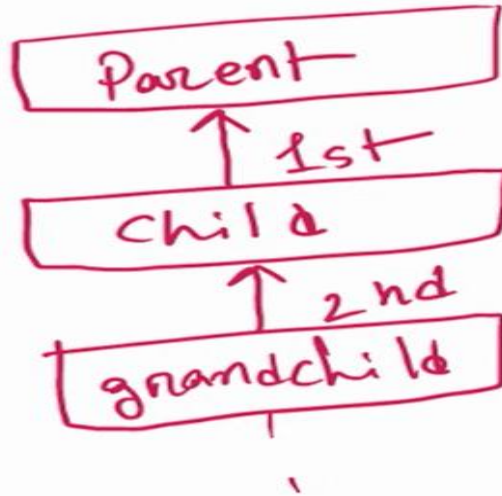
Baba abr nije banaise 10 bigha. Tahole chele pabe koto?

Shudhu 10 nki 2 . Nki puratai 12 bigha? Ans : 12 bigha e.

Baba er tao pabe.

Dada er tao pabe.

Multi level Inheritance

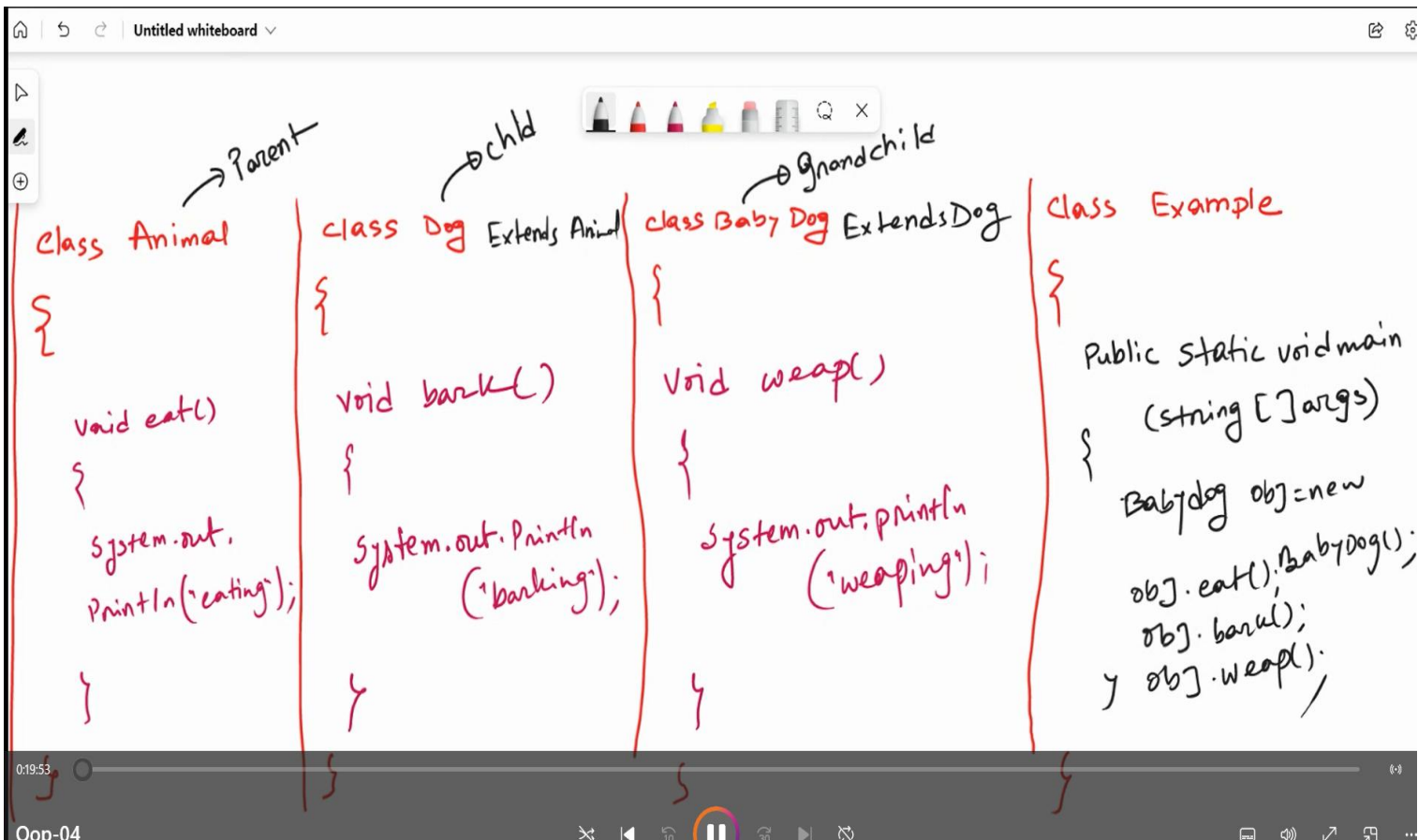


Dog is an animal. Baby dog is a child of dog. So animal parent, dog child, babydog is grandchild.

Untitled whiteboard

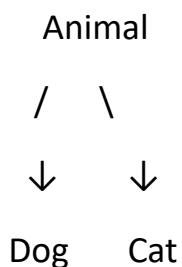
```
class Animal {  
    // ...  
}  
class Dog {  
    // ...  
}  
class BabyDog {  
    // ...  
}  
class Example {  
    // ...  
}
```

The whiteboard shows four class definitions separated by vertical red lines. Above the first line is the label 'Parent' with an arrow pointing to the 'class Animal' definition. Above the second line is the label 'child' with an arrow pointing to the 'class Dog' definition. Above the third line is the label 'Grandchild' with an arrow pointing to the 'class BabyDog' definition. The fourth line is 'class Example' with no label. Each class definition is enclosed in curly braces. The whiteboard interface includes a toolbar with drawing tools and a video player at the bottom.

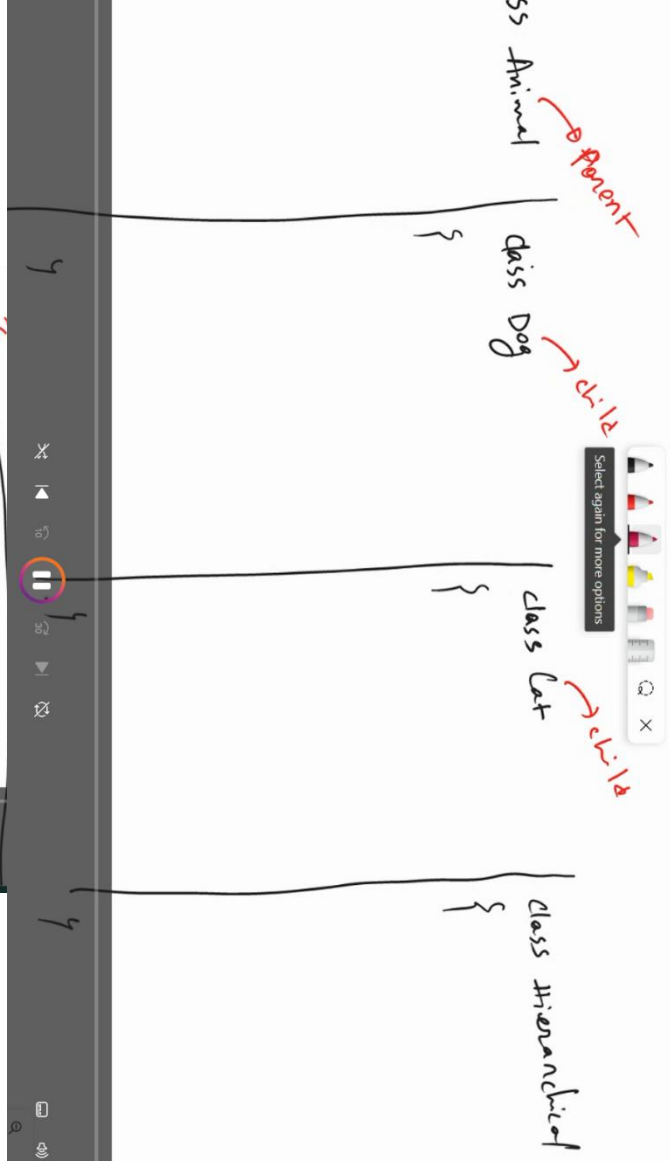


Hierarchical inheritance: When **multiple child classes inherit from the same parent class**, it is called **hierarchical inheritance**.

Dog is a animal. Cat is a animal. Both are child of animal class. Dog and cat are **siblings**.



```
class Animal
{
    void eat()
{
}
```



hierarchial

dic static void main
(string [] args)

```
Dog dog = new Dog();
dog.bark();
dog.eat();
```



Java

```
// Parent class
class Animal {

    String name;

    void eat() {
        System.out.println(name + " is eating.");
    }
}

// Child class
class Dog extends Animal {

    void bark() {
        System.out.println(name + " is barking.");
    }
}

// Child class (Sibling of Dog)
class Cat extends Animal {

    void meow() {
        System.out.println(name + " is meowing ");
    }
}
```



```
// Main class
public class Main {

    public static void main(String[] args) {

        Dog d1 = new Dog();

        d1.name = "Tommy";

        d1.eat();
        d1.bark();

        Cat c1 = new Cat();

        c1.name = "Kitty";

        c1.eat();
        c1.meow();

    }
}
```

Polymorphism: **Polymorphism** means "many forms."

In Java, it means **one method/object can behave in different ways** depending on the situation.

There are two main types:

1. **Compile-time polymorphism** → Method Overloading
2. **Run-time polymorphism** → Method Overriding

Runtime polymorphism: **Run-time polymorphism** means that the method to execute is decided **at run time**, based on the **actual object**, not the reference type.

```
// Parent class
class Animal {

    void sound() {
        System.out.println("Animal makes sound");
    }
}

// Child class
class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

// Child class
class Cat extends Animal {

    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
```

```
// Main class
public class Main {

    public static void main(String[] args) {

        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.sound();
        a2.sound();
    }
}
```

Output

★ Up

```
Dog barks  
Cat meows
```



Why is this called run-time polymorphism?

Look carefully:

```
</> Java  
Animal a1 = new Dog();
```



The reference is `Animal`, but the actual object is `Dog`.

So when Java sees:

```
</> Java  
a1.sound();
```



it checks the actual object at runtime:

```
a1  
↓  
Dog object  
↓  
Dog's sound()  
↓
```



↓
"Dog barks"

Similarly:

```
</> Java  
Animal a2 = new Cat();  
a2.sound();
```



becomes:

```
a2
↓
Cat object
↓
Cat's sound()
↓
"Cat meows"
```



The key formula

```
Parent reference
+
Child object
+
Overridden method
↓
Run-Time Polymorphism
```



For example:

```
</> Java
Animal a1 = new Dog();
```



is the important line.

If you instead write:



```
</> Java
Dog d1 = new Dog();
d1.sound();
```



the `Dog` method still runs, but this example does not demonstrate the key parent-reference/child-object form of runtime polymorphism.

Remember

Runtime polymorphism = Method Overriding + Parent reference referring to Child object.

Runtime polymorphism → Method signature.

Runtime polymorphism → Method signature → Same

class vehicle

```
{  
    void Accelerate()  
    {  
        System.out.println("Accelerating");  
    }  
}
```

class Bike extends vehicle

```
{  
    void Accelerate()  
    {  
        System.out.println("running");  
    }  
}
```

Compile-time polymorphism — Method name → Same
↳ same class

Class Addition

{

Public int Sum(inta, intb)

{ int c = a + b;

return c;

Public int sum(inta, intb, inte)

{ int d = a + b + c;

return d;

}

{

Compile-time polymorphism — Method name → Same

→ same class

Method
overloading

Class Addition

```
{  
    public int Sum(int a, int b)  
    {  
        int c = a + b;  
        return c;  
    }  
    public int sum(int a, int b, int c)  
    {  
        int d = a + b + c;  
        return d;  
    }  
}
```

Compile-time polymorphism — Method name → Same

→ same class

→ Same method
Name

→ return type same

Class Addition

```
{  
    public int (  
    {  
        int c = a
```

How it is still overloading possible?

The three types still overloading possible.

1. No of Parameter → $\left. \begin{array}{l} \text{Add}(\text{int}, \text{int}) \\ \text{Add}(\text{int}, \text{int}, \text{int}) \end{array} \right\} \checkmark$
2. Change in Data type → $\left. \begin{array}{l} \text{Add}(\text{int}, \text{int}) \\ \text{Add}(\text{double}, \text{double}) \end{array} \right\} \checkmark$
3. Change Sequence in Datatype → $\left. \begin{array}{l} \text{Add}(\text{int}, \text{double}) \\ \text{Add}(\text{double}, \text{int}) \end{array} \right\} \checkmark$

But the return type should be same.

Example 1: Different return types — ☒ Allowed

The parameters are different, so Java can distinguish them:

```
</> Java
class Test {

    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Here:

```
add(int, int)      → int
add(double, double) → double
```

☒ This is valid method overloading.

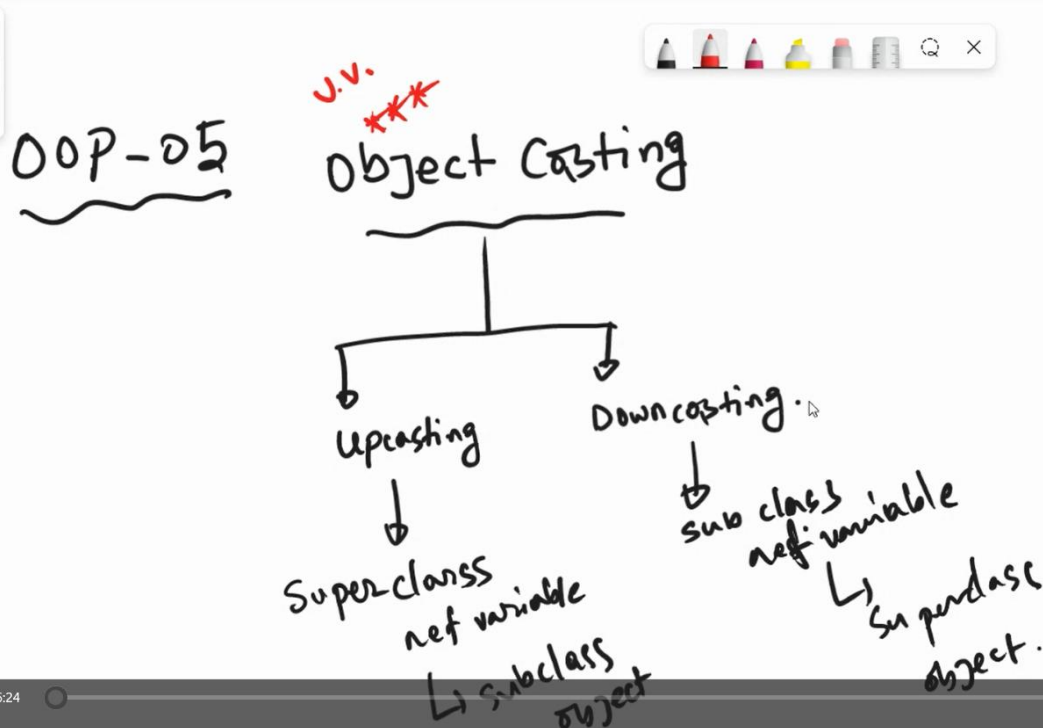
Example 2: Same parameters, different return type — ❌ Not allowed

```
Java  
class Test {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(int a, int b) {  
        return a + b;  
    }  
}
```

❌ Java gives an error because both methods have:

```
add(int, int)
```

Java doesn't consider the return type when deciding which overloaded method to call.



If method overriding happened, then if we do upcasting then child class method should be accessed.

Case: 1

<pre> Public class Parent { } </pre>	<pre> Public class child extends Parent { } </pre>	<pre> Public class Example { Public static void main (String [] args) { } } </pre>
--	--	--

If method overriding happened, then if we do upcasting then child class method should be accessed.

Case: 1

<pre> Public class Parent { void play() { System.out.println ("playing"); } } </pre>	<pre> Public class child extends Parent { void play() { System.out.println ("played"); } } </pre> ↳ playing	<pre> Public class Example { Public static void main (String [] args) { Parent P = new Parent(); P.play(); } } </pre>
--	--	---

Output: Playing

If method overriding happened, then if we do upcasting then child class method should be accessed.

Public class Parent	Public class child extends Parent	Public class Example
<pre> { void play() { System.out.println("playing"); } } </pre>	<pre> { void play() { System.out.println("played"); } } </pre> 1) playing 2) played.	<pre> { public static void main (String [] args) { Parent p = new child(); p.play(); } } </pre>

Output now: Playing

Case: 2: If method overriding not happened, then if we do upcasting then only parent class method can be accessed.

Public class Parent	Public class child extends Parent	Public class Example
<pre> { public void show() { System.out.println("showing"); } } </pre>	<pre> { public void play() { System.out.println("playing"); } } </pre>	<pre> { public static void main (String [] args) { Parent p = new child(); p.show(); } } </pre>

1. if in the picture in the main class Example

Parent p = new child();

p.show();

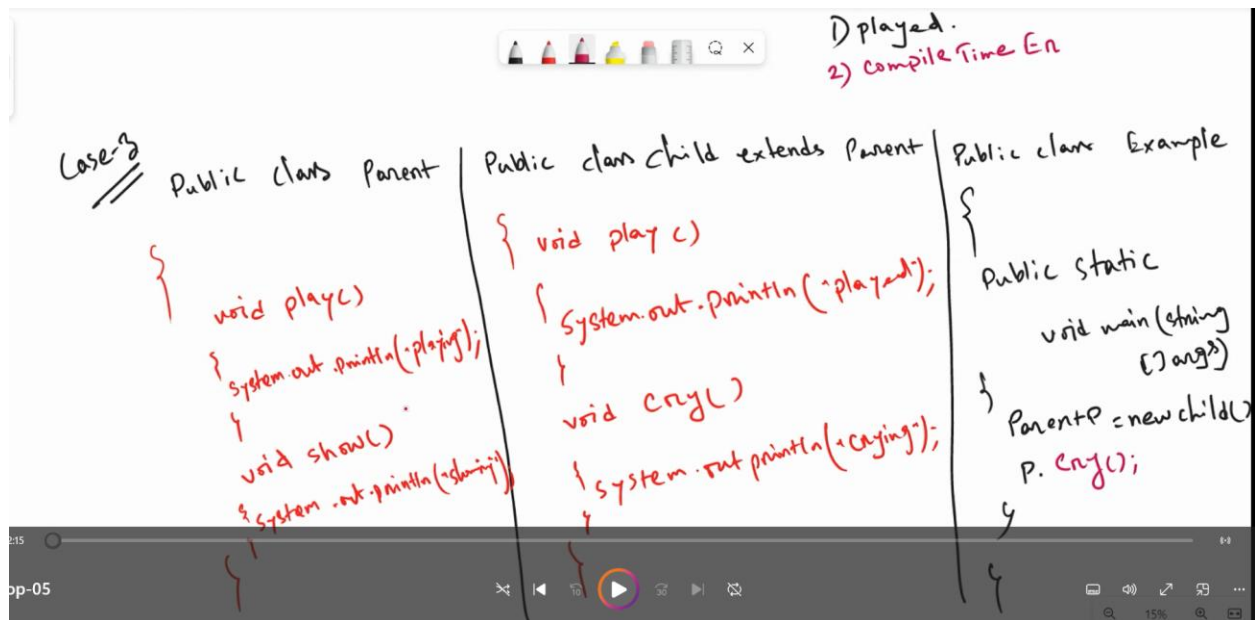
output: showing.

2. If in the picture in the main class

```
Parent p= new child();
```

```
p.play();
```

output: compile time error



1.if in the picture at is the main class Example

```
Parent p= new child();
```

```
p.play();
```

output: played;

2.If in the picture in the main class Example

```
Parent p= new child();
```

```
p.Cry();
```

output: compile time error

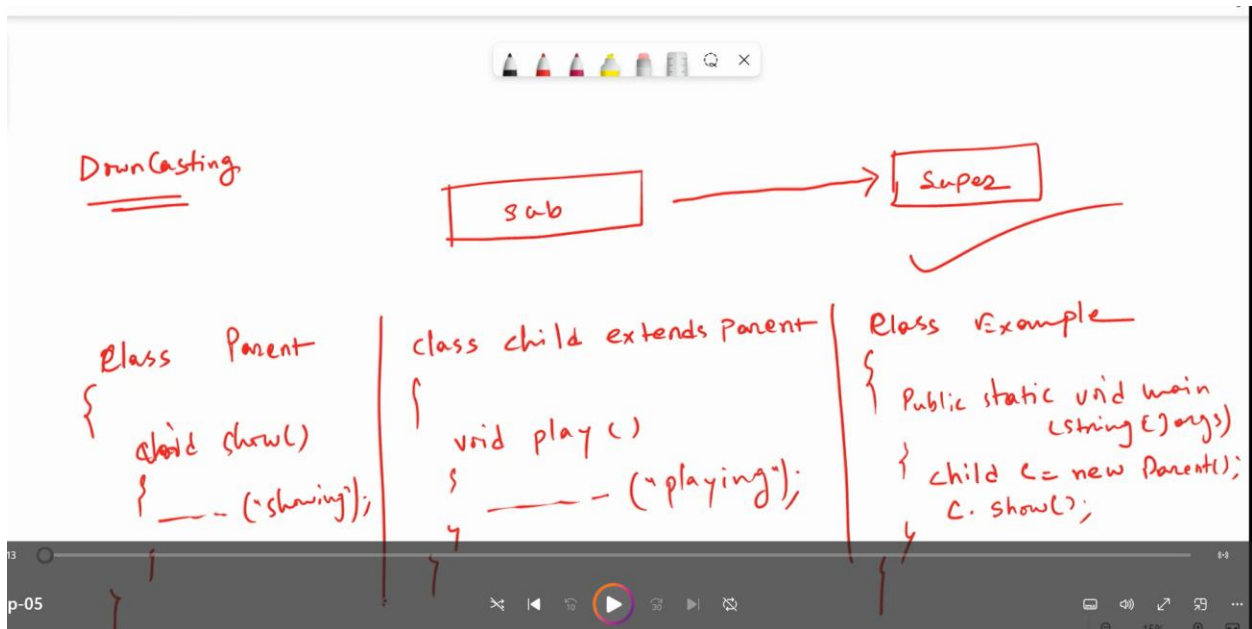
3. If in the picture in the main class Example

```
Parent p= new child();
```

```
p.show();
```

output: Showing

Only access the override methods or the parent class method.



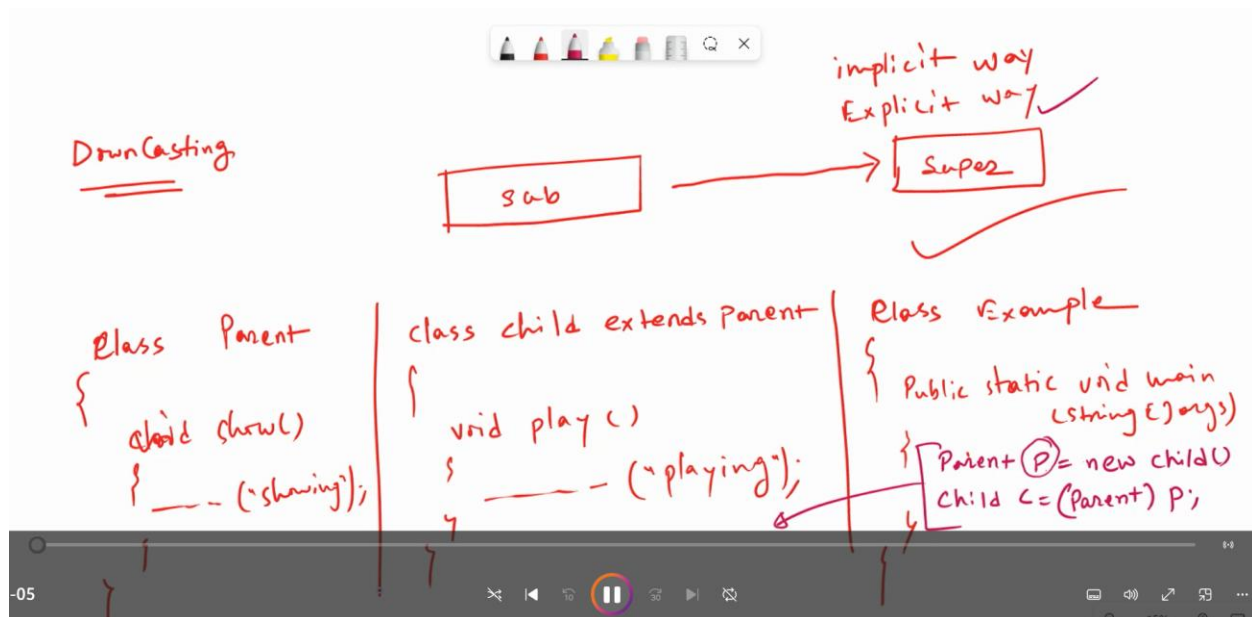
Output:

Runtime error.

Shows that exception .java doesn't support.

Implicit way.

If you want to access this down casting. U need to do the upcasting first.



In this case first parent class reference(p) created then new object child reference c. then we can access both parent and child class method.

Main class example{

Parent p=new child();

Child c=(Parent) p;

c.show();

c.play();

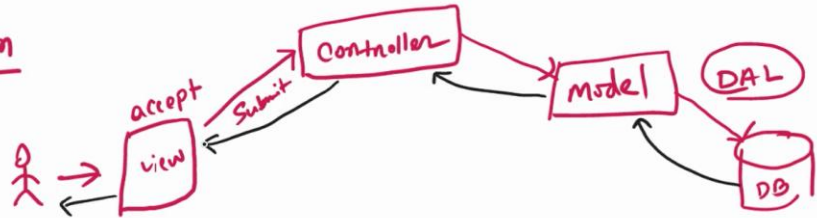
output:

showing

playing

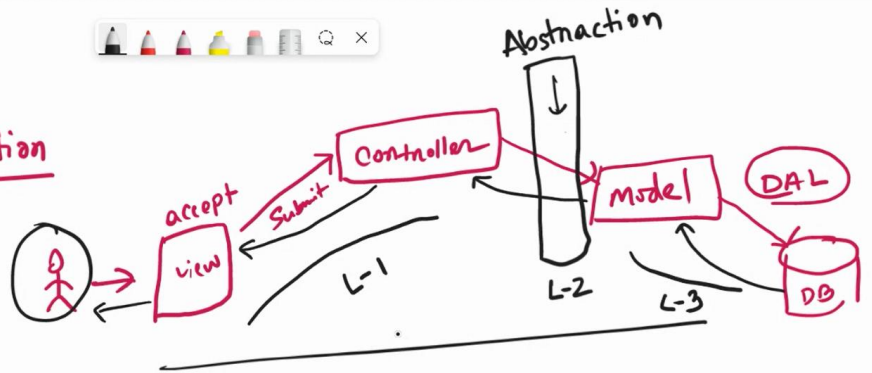
oop-6

Abstraction



oop-6

Abstraction



3 layer Architecture:

Abstraction:

Image baki ase.

Abstract class

- 'abstract' keyword.
- abstract method + non abstract method.
- It cannot be instantiated.
- It can have constructor, static method and final method.

Abstract method

- abstract class + Interface.
- abstract keyword
- It has no body in abstract class or Interface.
- The body is provided on the subclass.

abstract class Animal

```
{  
    public abstract void Sound();  
    public void sleep()  
{  
    system.out.println("sleeping");  
}
```

abstract

Non abstract

-06



10



30

abstract class Animal

```
{  
    public abstract void Sound();  
    public void sleep()  
{  
    system.out.println("sleeping");  
}
```

abstract

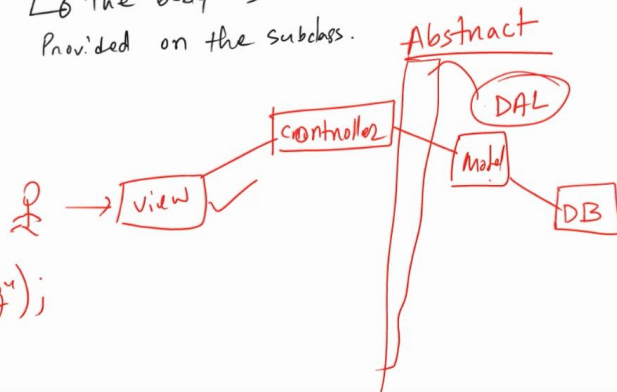
Non abstract

class Dog extends Animal

```
{  
    public void sound()  
{  
    system.out.println("sounding");  
}
```

Class or Interface.

The body is provided on the subclass.



10

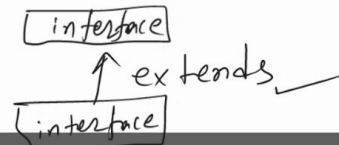
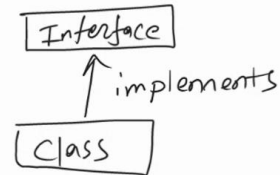
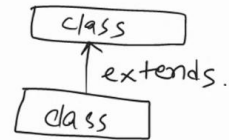


30



Interface

- reference type
- All are abstract method.



```
interface Printable
```

```
{
```

```
    void Print();
```

```
}
```

```
interface Showable
```

```
{
```

```
    void show();
```

```
}
```



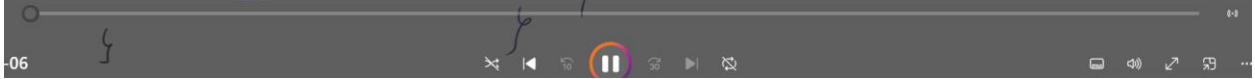
Multiple Inheritance
↓

```
interface Printable  
{  
    void Print();  
}
```

```
interface Showable  
{  
    void show();  
}
```

Class Example implements Printable, Showable

```
{  
    public void Print()  
    {  
        System.out.println("Hello");  
    }  
    public void show()  
    {  
        System.out.println("world");  
    }  
}
```



OOP-2

this
↳ reference variable

↳ refer

↳ current class object/method/
constructor variable.

this → Current class variable

```
class student
{
    int rollno;
    string name;
    float fee;
    student(int rollno, string name,
            float fee)
    {
        this.rollno = rollno;
    }
}
```

class example

```
{
    public static void main (string
                           [] args)
    {
    }
}
```

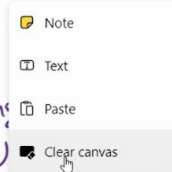
this → Current class variable

```
class student
{
    int rollno;
    string name;
    float fee;
    student(int rollno, string name,
            float fee)
    {
        this.rollno = rollno;
        this.name = name;
        this.fee = fee;
    }
}
```

Annotations:
 - rollno is circled with an arrow pointing to '1'.
 - name has an arrow pointing to 'x77'.
 - fee has an arrow pointing to '430.5'.
 - The constructor parameters have arrows pointing to their respective values: '1' for rollno, 'x77' for name, and '430.5' for fee.

class example

```
{
    public static void main (string
                           [] args)
    {
        student s = new student(1, 'x77', 430.5);
    }
}
```



this → refer current class method

class Test

{

void m1()

{ system.out.println("xyz");

}

void n1()

{ system.out.println("abc");

}

this → refer current class method

class Test

{

void m1()

{ system.out.println("xyz");

this.n1();

}

void n1()

{ system.out.println("abc");

}

xyz

abc

First print xyz then abc?

How to do this?

this → refer current class constructor:

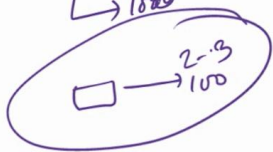
class Test

```
{  
    ✓ Test()  
    {  
        system.out.println("Test");  
    }  
    ✓ Test(int a, int b)  
    {  
        int c;  
        c = a + b;  
        ✓ this.test  
    }  
}
```

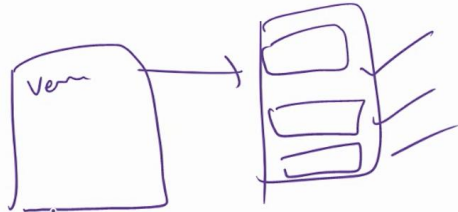
Encapsulation

2g-3g

↳ 1000



10000
↳



- 1) Control over data member
 - 2) data hiding
 - 3) Testing easier
- a) read only or write only

Public class Example

```
{  
    String Name;  
    int Age;  
    void Display()  
    {  
        System.out.println("Name" + Name + "Age" + Age);  
    }  
}
```

→ variable

Method

Public class Example

```
{  
    ✓ String Name;  
    ✓ int Age;  
    ✓ void Display()  
    {  
        ✓ System.out.println("Name" + Name + "Age" + Age);  
    }  
}
```

→ variable

Method

Public class Test

```
{  
    public static void main(String[] args)  
    {  
        Example x = new Example();  
        x.Name = "xyz";  
        x.Age = 27;  
        x.Display();  
    }  
}
```

Public class Example

```

{
    Private String Name;
    Private int Age;

    Public void setName(String Name)
    {
        this.Name = Name;
    }

    Public void getName()
    {
        return Name;
    }

    Public void setAge(int Age)
    {
        this.Age = Age;
    }

    Public int GetAge()
    {
        return Age;
    }
}

```

Setter

Getter

0:42:55

Oop-07

Getter should be
public String GetName()

{

return Name;

}

Public class Example

```

{
    Private String Name;
    Private int Age;

    Public void setName(String Name)
    {
        this.Name = Name;
    }

    Public void getName()
    {
        return Name;
    }

    Public void setAge(int Age)
    {
        this.Age = Age;
    }

    Public int GetAge()
    {
        return Age;
    }
}

```

Setter

Getter

Public class EncompTest

```

{
    Public static void main
        (String[] args)
    {
        Example x = new Example();
        x.setName("xyz");
        x.Age(29);
        System.out.println
            (x.getName() +
             x.GetAge());
    }
}

```

Getter should be
public String GetName()

{

return Name;

}

Has a Relationship:

Employee has a Address.

The image shows a handwritten note on a whiteboard titled "Aggregation" with a double underline. It defines two classes: `Address` and `Employee`. The `Address` class is written in purple and includes attributes `city`, `state`, and `country`, along with a constructor and a setter. The `Employee` class is written in red and is currently empty. To the right of the code, there are two hand-drawn boxes: one labeled "Address" and another labeled "Employee", each with a checkmark to its left. The entire content is captured within a video player interface, with a toolbar at the bottom.

Aggregation

```
public class Address
{
    string city, state, country;
    Address (string city, String state, string country)
    {
        this.city = city;
        this.state = state;
        this.country = country;
    }
}
```

✓ Address

✓ Employee

Public class Employee

negation

```
Public class Address ✓  
{  
    string city, state, country;  
    Address (string city, String  
            state, string country) {  
        { this.city = city;  
          this.state = state;  
          this.country = country;  
        }  
    }  
}
```

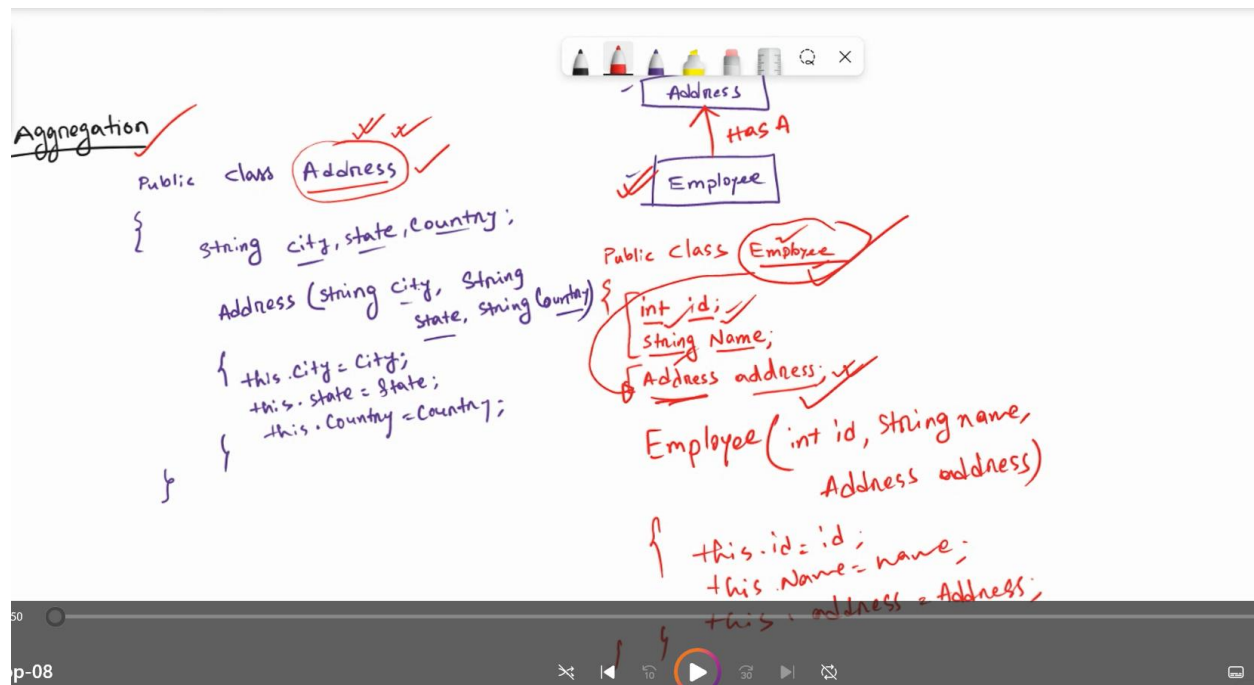
Public Class Employee ✓

```
int id;  
string Name;  
Address address;
```

int, String is a class, and it is also a default datatype.

Address is a class, it is also a userdefined.

This is called aggregation.



Aggregation

```

Public class Address
{
    String city, state, country;
    Address (String city, String state, String country)
    {
        this.city = city;
        this.state = state;
        this.country = country;
    }
}
    
```



```

Public class Employee
{
    int id;
    String Name;
    Address address;
    Employee (int id, String name, Address address)
    {
        this.id = id;
        this.Name = name;
        this.address = address;
    }
}
    
```

p-08

Aggregation

```

Public class Address
{
    String city, state, country;
    Address (String city, String state, String country)
    {
        this.city = city;
        this.state = state;
        this.country = country;
    }
}
    
```

```

Public class Employee
{
    int id;
    String Name;
    Address address;
    Employee (int id, String name, Address address)
    {
        this.id = id;
        this.Name = name;
        this.address = address;
    }
}
    
```

Rakib
112
Rakib city, state
Dinajpur, Bangladesh

Public class Example

```

{
    Public static void
    main (String args[])
    {
        Address a = new Address
        ("Dinajpur", "Dinajpur",
        "BD");
        Employee e = new Employee
        (112, "Rakib", a);
    }
}
    
```

Aggregation

Public class Address

```

{
    String city, state, country;
    Address (String city, String state, String country)
    {
        this.city = city;
        this.state = state;
        this.country = country;
    }
}

```

Rakib
112
city Dinajpur, State Bangladesh

Public class Employee

```

{
    int id;
    String Name;
    Address address;
    Employee (int id, String name, Address address)
    {
        this.id = id;
        this.Name = name;
        this.address = address;
    }
}

```

Address

Employee

Has A

Public class Example

```

{
    Public static void main (String [] args)
    {
        Address a = new Address ("Dinajpur", "Dinajpur", "BD");
        Employee e = new Employee (112, "Rakib", a);
        System.out.println (e.address.city);
    }
}

```

Oop-08

Exception handling:

Error: Syntax Error / Compile time error

class Test

```

{
    Public static void main (String [] args)
    {
        int x;
        x = 12/0;
        System.out.println (x);
    }
}

```

Exception: logical error/ Runtime error

```
class Test
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int x;
```

```
        x = 12/0;
```

```
        System.out.println(x);
```

```
    }
```

```
}
```

```
int n; i;  
n = 12/0;  
System.out.println(n);  
for(i=0; i<10; i++)  
{  
    System.out.println(i);  
}
```

X=12/0;

Need to do exception handling.

Possibility of error which should be in try{} block.

Similarly, which one will be wrong for that we need catch{} block.

Class Test

```
{ public static void main (String [] args)
```

```
{ int n, i;
```

```
try
```

```
{  
→ n = 12/0;
```

```
  System.out.println(n);
```

```
}
```

```
catch (Exception ex)
```

```
{  
  System.out.println(ex);
```

```
}
```

Class Test

```
{ public static void main (String [] args)
```

```
{ int n, i;
```

```
try
```

```
{ n = 12/0;
```

```
  System.out.println(n);
```

```
} catch (Exception ex)
```

```
{ System.out.println(ex);
```

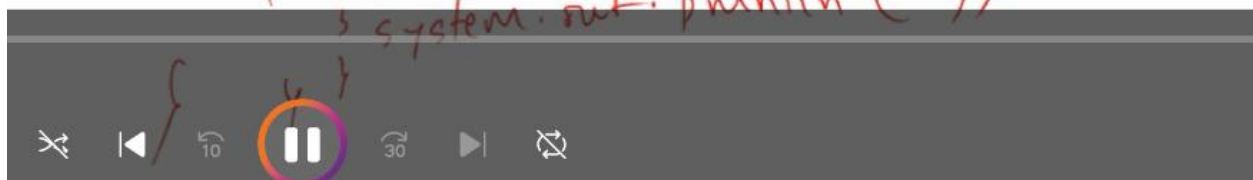
```
}
```

```
finally
```

```
{ for (i = 0; i < 10; i++)
```

```
  System.out.println(i);
```

```
}
```



finally is a block of code that is executed after try and catch, whether an exception occurs or not.

It is mainly used for **cleanup operations**, such as closing files, database connections, or other resources.

Basic structure

```
</> Java   
  
try {  
    // risky code  
}  
catch (Exception e) {  
    // handles exception  
}  
finally {  
    // always executes  
}
```

Example 1: Exception occurs

```
</> Java   
  
public class Main {  
    public static void main(String[] args) {  
  
        try {  
            int x = 10 / 0;  
        }  
        catch (Exception e) {  
            System.out.println("Exception occurred");  
        }  
        finally {  
            System.out.println("Finally executed");  
        }  
    }  
}
```



Output

Exception occurred
Finally executed



Example 2: No exception

 Java



```
try {  
    int x = 10 / 2;  
    System.out.println(x);  
}  
catch (Exception e) {  
    System.out.println("Exception occurred");  
}  
finally {  
    System.out.println("Finally executed");  
}
```

Output:

5
Finally executed



Why do we need `finally`?

Suppose you open a file:

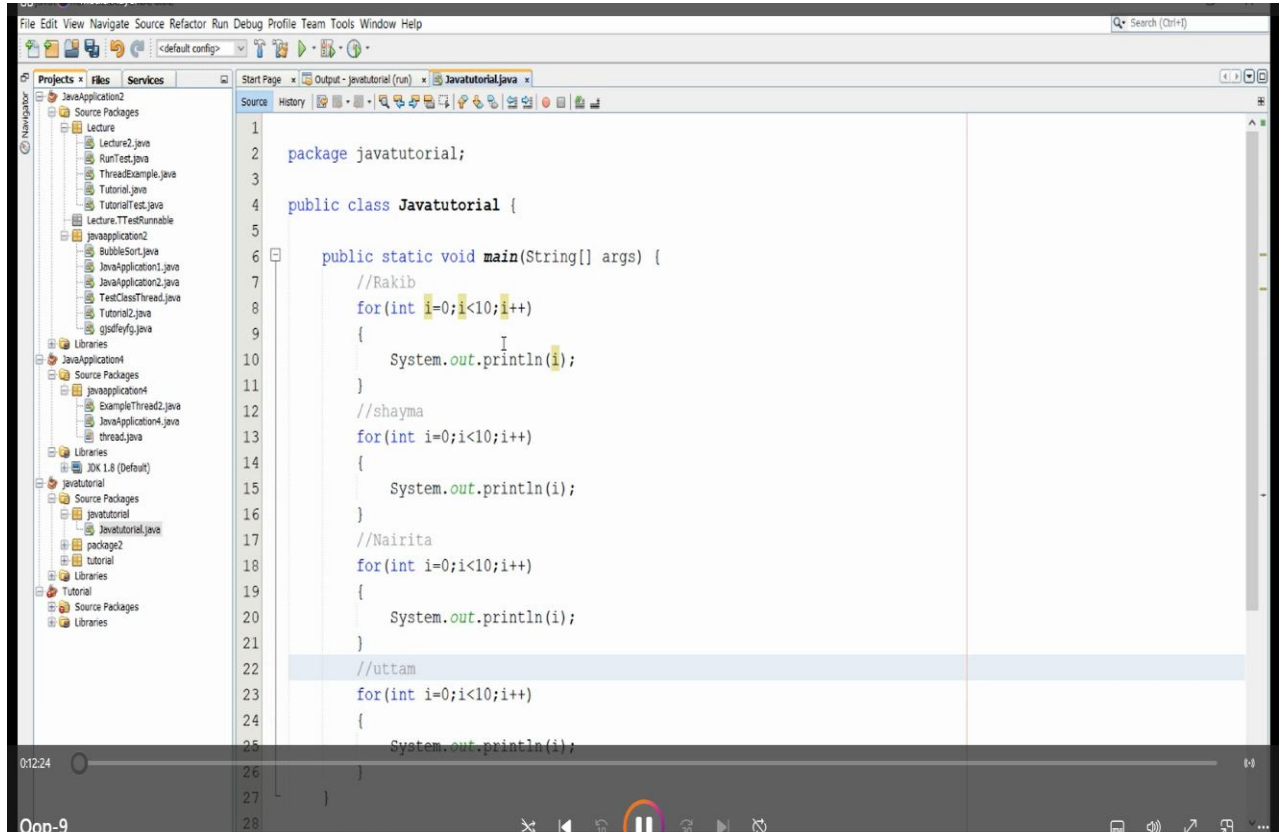
```
</> Java

try {
    // open file
    // read file
}
catch (Exception e) {
    // handle error
}
finally {
    // close file
}
```

Whether reading the file succeeds or fails, you want to close the file.

★ Easy way to remember

```
try    → Try the risky operation
catch  → Handle the exception
finally → Do the cleanup
```



File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

Search (Ctrl+F)

Projects | Files | Services

Start Page | Output - javatutorial (run) | Javatutorial.java

Source | History

Navigator

- JavaApplication2
 - Source Packages
 - Lecture
 - Lecture2.java
 - RunTest.java
 - ThreadExample.java
 - Tutorial.java
 - TutorialTest.java
 - Lecture.TTestRunnable
 - javaapplication2
 - BubbleSort.java
 - JavaApplication1.java
 - JavaApplication2.java
 - TestClassThread.java
 - Tutorial2.java
 - gsdfefg.java
 - Libraries
 - JavaApplication4
 - Source Packages
 - JavaApplication4
 - ExampleThread2.java
 - JavaApplication4.java
 - thread.java
 - Libraries
 - JDK 1.8 (Default)
 - javatutorial
 - Source Packages
 - javatutorial
 - package2
 - tutorial
 - Libraries
 - Tutorial
 - Source Packages
 - Libraries

```
1 package javatutorial;
2
3
4 public class Javatutorial {
5
6     public static void main(String[] args) {
7         //main thread
8     }
9
10
11 }
12
13
14 class A extends Thread
15 {
16     @Override
17     public void run()
18     {
19         for(int i=0; i<10;i++)
20         {
21             System.out.println("1");
22         }
23     }
24 }
25 }
```

0:29:28

Oop-9

javatutorial.A run for (int i=0; i<10; i++)

22:13 1/5

```
import java.util.logging.Level;
import java.util.logging.Logger;

public class Javatutorial {

    public static void main(String[] args) {
        //main thread
        A obj=new A();
        obj.start();
    }
}
```

One is thread . User defined thread.

```
public class Javatutorial {

    public static void main(String[] args) {
        //main thread
        A obj=new A();
        obj.start();
        for(int i=0; i<10;i++)
        {
            System.out.println("2");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                Logger.getLogger(A.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }

}

class A extends Thread
{
    @Override
    public void run()
    {
        for(int i=0; i<10;i++)
        {
            System.out.println("1");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                Logger.getLogger(A.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }
}
```

Multi thread: one is on the the public static side.

```
class A extends Thread
{
    @Override
    public void run()
    {
        for(int i=0; i<10;i++)
        {
            System.out.println("1");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                Logger.getLogger(A.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }
}
```

We can do this using interface.

```

7 public class ThreadExample {
8     public static void main(String[] args) {
9         B obj=new B();
10        Thread t1=new Thread(obj);
11        t1.start();
12
13
14        for(int i=0;i<10;i++)
15        {
16            System.out.println("2");
17            try {
18                Thread.sleep(1000);
19            } catch (InterruptedException ex) {
20                Logger.getLogger(B.class.getName()).log(Level.SEVERE, null, ex);
21            }
22        }
23    }
24
25 }

```

```

class B extends C implements Runnable
{
    public void run()
    {
        for(int i=0;i<10;i++)
        {
            System.out.println("1");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException ex) {
                Logger.getLogger(B.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
    }
}

class C
{
}

```

Problem: 2

```
public class Base
{
    private int multiplier(int data)
    {
        return data*5;
    }
}

class Derived extends Base
{
    private static int data;
    public Derived()
    {
        data = 25;
    }
    public static void main(String[] args)
    {
        Base temp = new Derived();
        System.out.println(temp.multiplier(data));
    }
}
```

125

Output

Compile time Error

Integer
↳
-128 to 127

```
package OutputProblem;
public class Problem49 {
    public static void main(String[] args) {
        Integer a = 128, b = 128;
        System.out.println(a == b);
        Integer c = 100, d = 100;
        System.out.println(c == d);
    }
}
```

true
true

→ false

→ true ✓

Output: False True

False : unsigned int range -128 to 127 that's why it is false.

True

```
class Base {  
    protected void foo() {}  
}  
class Derived extends Base {  
    void foo() {}  
}  
public class Main {  
    public static void main(String args[]) {  
        Derived d = new Derived();  
        d.foo();  
    }  
}
```

Public/default

The **base** class(parent class) method is protected. Then extends **derived** (child class) method is default which is more secure or less accessible then protected access modifier so it will not override the foo(). if the child class method is protected or more accessible than the method is overridable.

```
class Base {  
    protected void foo() {}  
}
```

```
class Derived extends Base {
```

```
    @Override
```

```
    protected void foo() {} // Fixed: Now matches the visibility of Base or
```

}

```
package main;
class Point {
    protected int x, y;
    public Point(int _x, int _y)
    {
        x = _x;
        y = _y;
    }
}
public class Main {
    public static void main(String args[])
    {
        Point p = new Point();
        System.out.println("x = " + p.x + ", y = " + p.y);
    }
}
```

Compile Time Error

Output:

Compile time error.

The constructor is parameterized constructor. So the object should be

Point p= new Point(10,1);

But the question is passing no value in the object .

t

```
public class Test
{
    try
    {
        public Test()
        {
            System.out.println("GeeksforGeeks");
            throw new Exception();
        }
    }
    catch(Exception e)
    {
        System.out.println("GFG");
    }
    public static void main(String[] args)
    {
        Test test = new Test();
    }
}
```

output: compile time error.

```
interface OuterInterface
{
    public void InnerMethod();
    public interface InnerInterface
    {
        public void InnerMethod();
    }
}

public class Outer implements OuterInterface.InnerInterface, OuterInterface
{
    public void InnerMethod()
    {
        System.out.println(100);
    }

    public static void main(String[] args)
    {
        Outer obj = new Outer();
        obj.InnerMethod();
    }
}
```

Nested.
Multiple Inheritance.
100

Output:

100

Kon interface er jonno 100 value set hoise?

a. inner interface b. outer interface c. both d. none

ans: both

Nested Interfaces are Implicitly Static : In Java, any interface defined inside another interface (like InnerInterface inside OuterInterface) is automatically **public and static**. This is why referencing it as OuterInterface.InnerInterface is completely valid.

Method Signature Merging: The class Outer implements two different interfaces, both containing a method named InnerMethod() with the exact same signature (public void InnerMethod()). In Java, if a class

implements multiple interfaces that share an identical method signature, **a single implementation satisfies both interfaces.**

Execution: Inside the main method, an instance of Outer is created, and calling obj.InnerMethod() executes the single overridden method, printing 100.

```
package OutputProblem;
public class Problem44 {
    public static void main(String[] args) {
        try
        {
            System.out.println("First statement of try block");
            int num=45/3;
            System.out.println(num); → 15
        }
        catch(Exception e)
        {
            System.out.println("Gfg caught Exception");
        }
        finally
        {
            System.out.println("finally block"); → ✓
        }
        System.out.println("Main method"); → ✓
    }
}
```

Output:

First semester of Try block

15

finally block

Main method

```
package OutputProblem;
public class Problem44 {
    public static void main(String[] args) {
        try
        {
            System.out.println("First statement of try block");
            int num=45/3; 45/0
            System.out.println(num);
        }
        catch(Exception e)
        {
            System.out.println("Gfg caught Exception");
        }
        finally
        {
            System.out.println("finally block");
        }
        System.out.println("Main method");
    }
}
```

Output:

First semester of try block

Gfg caught Exception

finally block

Main method

```

package OutputProblem;
public class Problem27 {
    Test1 t1 = new Test1(10);
    Problem27 (int i)
    {
        t1 = new Test1(i);
    }
    public static void main(String[] args) {
        Problem27 t2 = new Problem27(5);
    }
}
class Test1 {
    Test1(int x) {
        System.out.println("Constructor called " + x);
    }
}

```

Output:

```

package OutputProblem;
public class Problem27 {
    Test1 t1 = new Test1(10);
    → Problem27 (int i)
    {
        t1 = new Test1(i);
    }
    public static void main(String[] args) {
        Problem27 t2 = new Problem27(5);
    }
}
class Test1 {
    Test1(int x) {
        System.out.println("Constructor called " + x);
    }
}

```

Constructor called 10
 Constructor called 5

